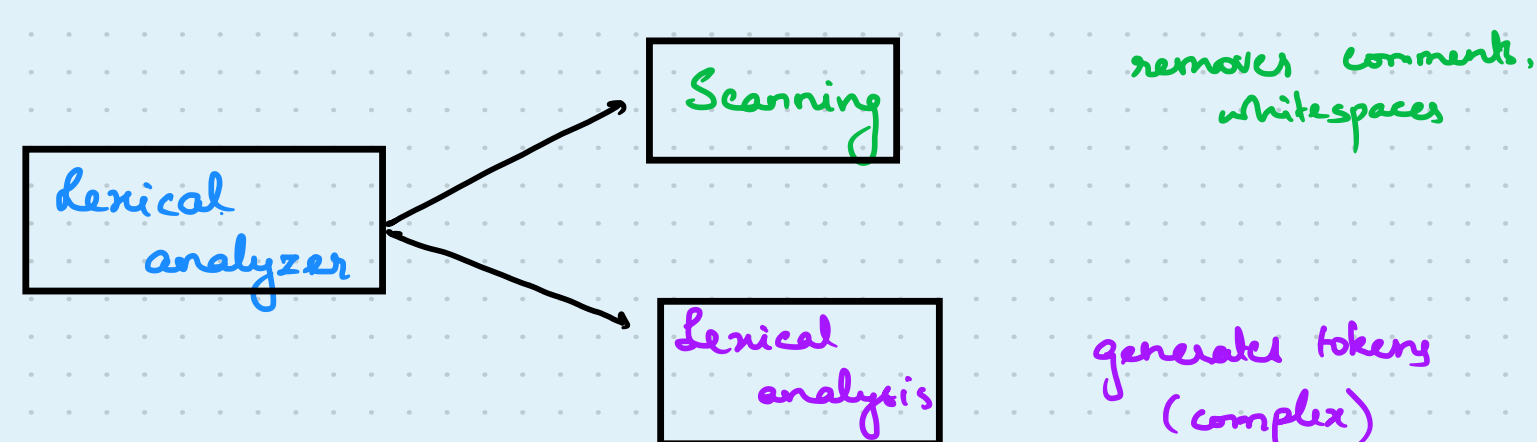


Pass

- group of one or more phases of a compiler that does analysis/synthesis of source program

LEXICAL ANALYZER / LEXER / SCANNER

- first phase, language dependent



NOTE: Definitions

lexeme: actual sequence of chars in source

pattern: rule (often regex) defining valid char strings for a given token

token: category given to sequence (lexeme)

- Token $\rightarrow$ associated with lexeme
 - $\rightarrow$ optional attributes like numeric value
- Identifiers added to symbol table

NOTE: Attributes

$\langle \text{token}, \underline{\text{attribute}} \rangle$

$\hookrightarrow$ critical later

Usually only one attr, but could represent more info.

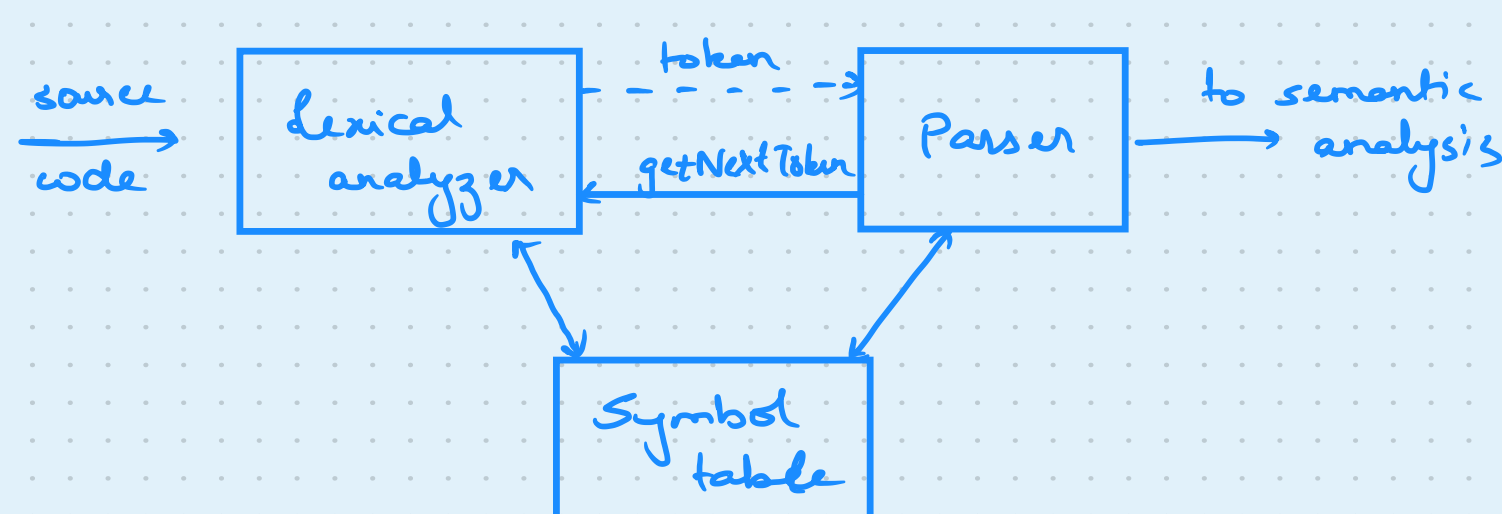
Identifiers: need type, location, value etc.

$\Rightarrow$ attribute = pointer to symbol table entry

Secondary tasks:

- Removing comments, whitespaces
- Correlating error messages with source
- Macro expansion
- Constructing source with error messages added at given locations

Dependence on parser:



Function names = identifiers $\neq$ keywords

Syntactic sugar

Syntax within a programming language that is designed to make things easier to read or express

Eg:

$\text{arr}[i] \iff *(&\text{arr} + i)$

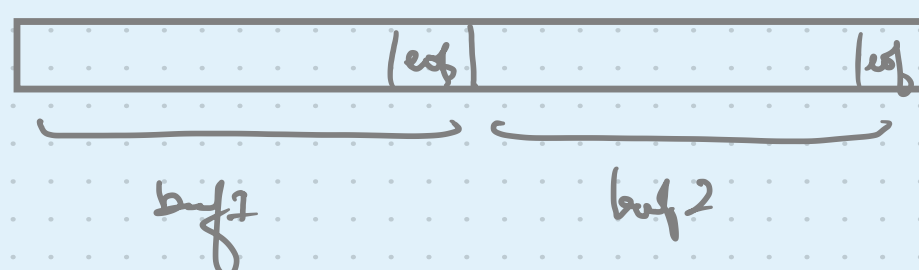
$x += i \iff x = x + i$

$\text{var } x = \text{expression} \quad // \text{compiler auto-assigns type [C\#]}$

desugaring: sugared statement $\rightarrow$ fundamental statement

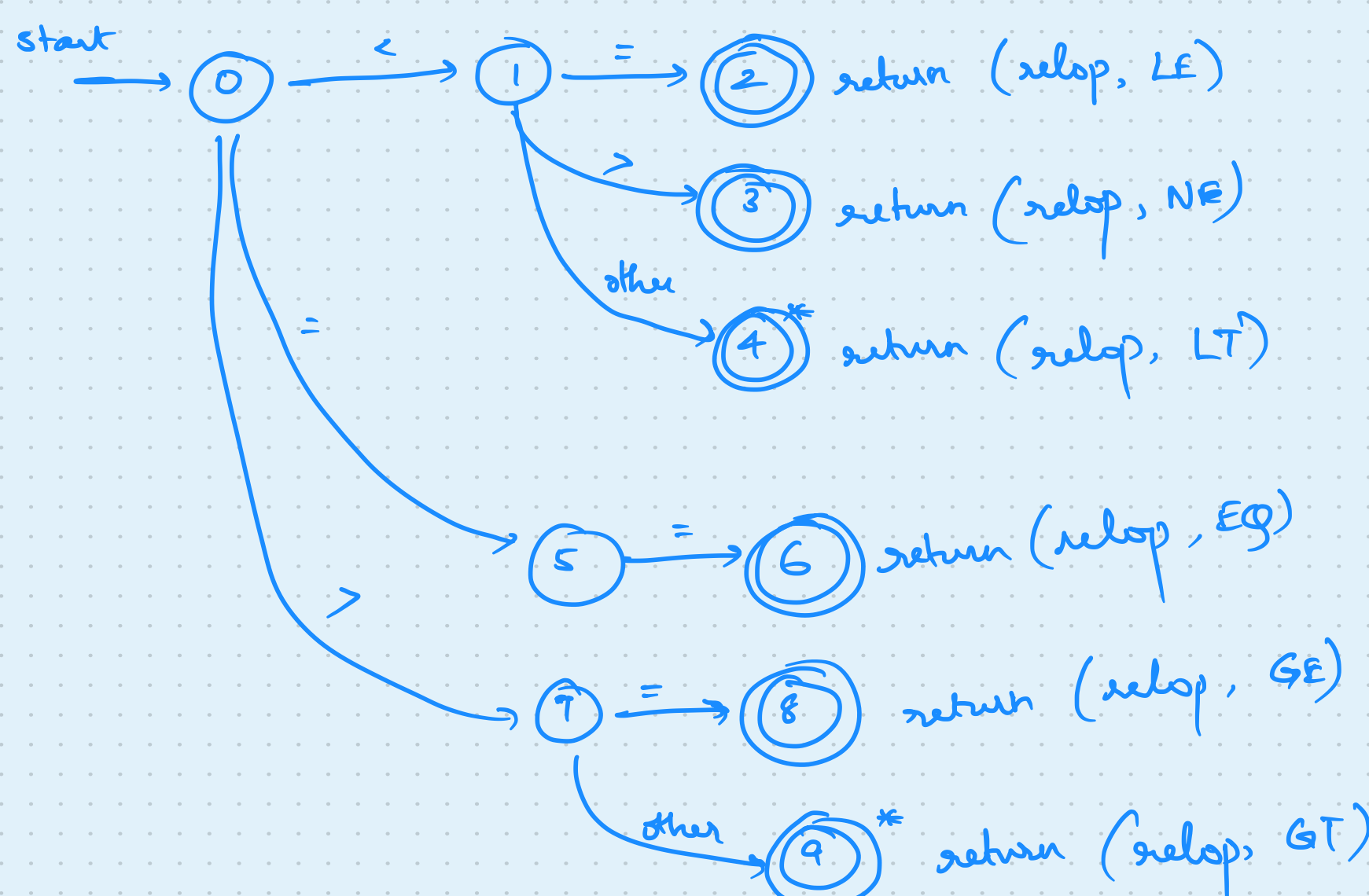
BUFFER PAIRS

- load 2 buffers of size $N \rightarrow$ System need to fill N chars in buffer instead of char by char
- Two pointers: lexemeBegin, forward
 - Move forward forward until pattern match is found
 - Retract forward by 1; $[\text{lexemeBegin} \dots \text{forward}] = \text{lexeme}$
 - Set forward and lexemeBegin to next char.
- When one buffer full $\rightarrow$ load the other, move forward to other buffer
- How to check? Sentinel [EOF] } special char that cannot show up in source prog
EOF at end of buf = reload other
EOF elsewhere = end of input



Regular expressions and Transition Diagrams

relop $< | <= | > | >= | == | <>$



Handling keywords

- Install them in symbol table ahead of time
 $\text{return}(\text{getToken}, \text{installID})$
- Separate transition diagrams for each keyword at higher priority than ID